

SESSION 02 / 10

基本構造と 最初の3Dシーン

Scene ・ Camera ・ Renderer で立方体を動かす

Week 2

2 hours

本日のアジェンダ

01

3つの構成要素を理解する

Scene / Camera / Renderer の役割

30 min

02

メッシュとループの仕組み

Mesh = Geometry + Material / アニメーションループ

30 min

03

ハンズオン演習

回転する立方体を画面中央に表示する

50 min

解説 60 min → ハンズオン 50 min → 振り返り 10 min

前提知識・準備物



第1回の到達点

- Vite プロジェクトを起動できる
- `npm install three` が完了
- ES Modules で `import` できる
- 青いキャンバスを表示できた



今回必要なもの

前回のプロジェクト

`my-threejs-app` をそのまま使用

ターミナル

`npm run dev` で起動できる状態

数学の知識

不要（コピペで進められる）

3Dシーンを支える3つの要素



Scene

シーン

3D空間そのもの。
オブジェクトを
配置する入れ物



Camera

カメラ

どこから・どの範囲を
見るかを定める
視点



Renderer

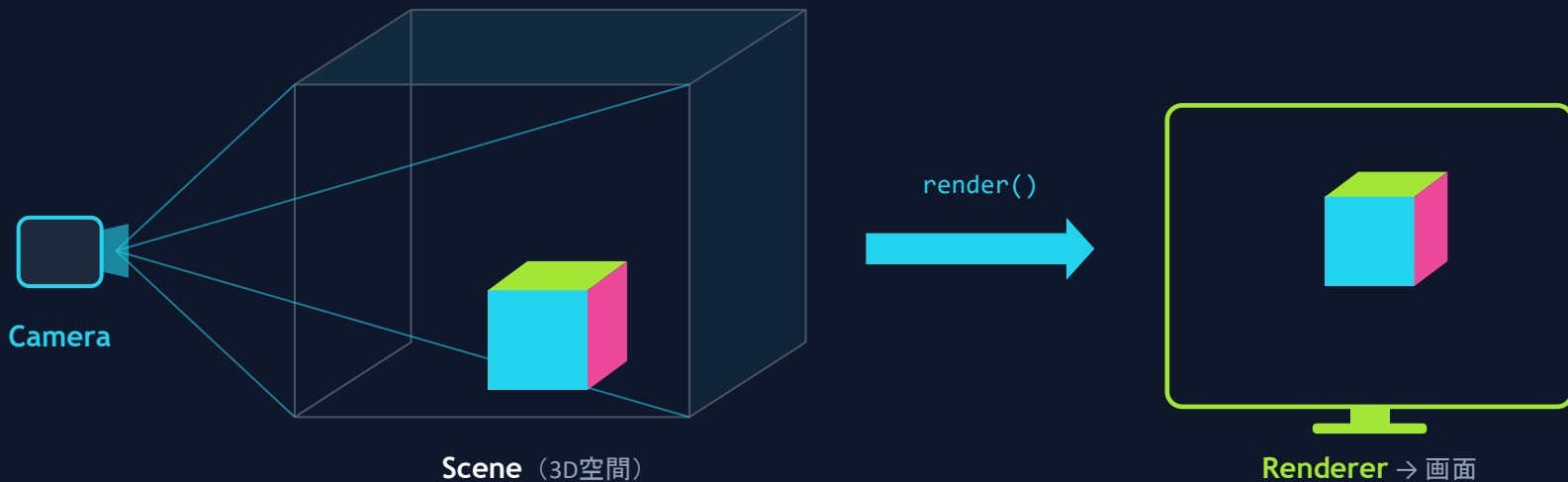
レンダラー

シーンをカメラ視点で
画面に描き出す
描画エンジン

この3つが揃って初めて、画面に3Dが表示される

3D空間をカメラが捉え、画面に映す

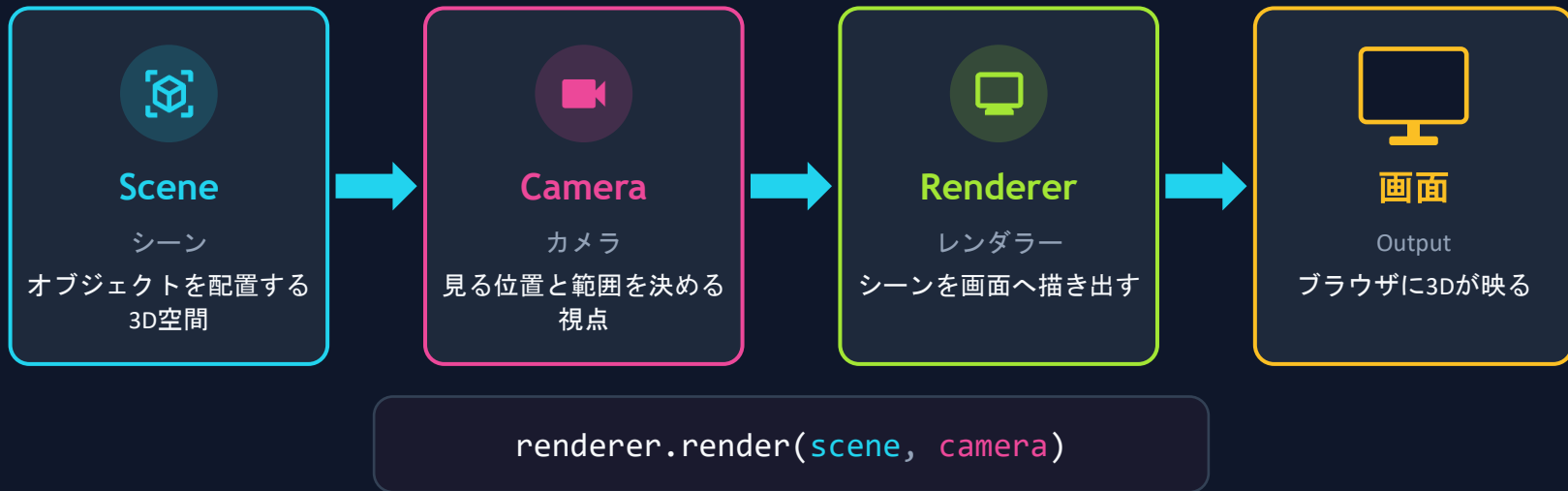
Camera の視野（フラスタム）が Scene を捉え、Renderer が2D画面へ投影する



3D空間（Scene）を Camera の視野が捉え、Renderer が2D画面へ描き出す

3要素が連携して画面に映る

オブジェクトを配置し、視点で捉え、画面へ描き出すまでの流れ



Scene に置いたオブジェクトを、Camera の視点で Renderer が画面へ描き出す

Scene — 3D空間の入れ物

すべてのオブジェクト・ライト・カメラを格納するルートコンテナ

```
import * as THREE from 'three';  
const scene = new THREE.Scene();  
scene.background = new THREE.Color(0xf172a);
```

→ **add() で追加**

scene.add(mesh) でオブジェクトを空間に配置

→ **階層構造**

親子関係（グループ）で複数まとめて管理できる

→ **背景・霧**

background や fog で空間の雰囲気を設定

PerspectiveCamera — 視点

人間の目に近い遠近感のあるカメラ。4つの引数で視界を定義

```
const camera = new THREE.PerspectiveCamera(  
  75, window.innerWidth / window.innerHeight, 0.1, 1000 );  
camera.position.z = 5;
```

fov**視野角（度）**

75 が標準的

aspect**アスペクト比**

幅 ÷ 高さ

near**最近接距離**

これより手前は非表示

far**最遠距離**

これより奥は非表示

WebGLRenderer — 描画エンジン

Scene をカメラ視点で <canvas> に描き出す心臓部

```
const renderer = new THREE.WebGLRenderer({ antialias: true });
renderer.setSize(window.innerWidth, window.innerHeight);
renderer.setPixelRatio(window.devicePixelRatio);
document.body.appendChild(renderer.domElement);
```

- `setSize()` 描画領域のピクセルサイズを指定
- `setPixelRatio()` Retina など高解像度ディスプレイに対応
- `render()` `renderer.render(scene, camera)` で1フレーム描画

PART 02

オブジェクトを作る

Mesh = Geometry + Material

Mesh = Geometry + Material



Geometry

形状

頂点の集まり
(立方体・球など)

+



Material

材質

表面の見た目
(色・質感)

=



Mesh

メッシュ

形状+材質を
合わせた3D物体

```
const mesh = new THREE.Mesh(geometry, material); scene.add(mesh);
```

代表的なジオメトリ

BoxGeometry

立方体・直方体

(1, 1, 1)

SphereGeometry

球体

(1, 32, 16)

PlaneGeometry

平面（床・壁）

(10, 10)

CylinderGeometry

円柱・円錐

(1, 1, 2)

TorusGeometry

ドーナツ形

(1, 0.4)

ConeGeometry

円錐

(1, 2, 32)

今回使うマテリアル

第3回で詳しく扱う。今回はまず動かすことを優先

MeshBasicMaterial

ライト不要で常に一定の色。
動作確認に最適

MeshNormalMaterial

面の向きを色で可視化。
デバッグに便利

MeshStandardMaterial

物理ベースのリアルな質感。
ライトが必要

```
const material = new THREE.MeshNormalMaterial();
```

最小コードで立方体を表示

```
import * as THREE from 'three';

const scene = new THREE.Scene();
const camera = new THREE.PerspectiveCamera(75, innerWidth/innerHeight, 0.1, 1000);
camera.position.z = 5;

const renderer = new THREE.WebGLRenderer({ antialias: true });
renderer.setSize(innerWidth, innerHeight);
document.body.appendChild(renderer.domElement);

const geometry = new THREE.BoxGeometry(1, 1, 1);
const material = new THREE.MeshNormalMaterial();
const cube = new THREE.Mesh(geometry, material);
scene.add(cube);

renderer.render(scene, camera);
```

描画されるまでの流れ

1

Scene を作る

オブジェクトを置く空間を用意

2

Camera を置く

どこから見るかを定める

3

Mesh を add

形状+材質をシーンに追加

4

Renderer で描画

render(scene, camera) を呼ぶ

ただし render を1回呼ぶだけでは静止画。動かすにはループが必要 →

アニメーションループ

requestAnimationFrame で毎フレーム再描画し、動きを生む

```
function animate() {  
  requestAnimationFrame(animate); // 次のフレームを予約  
  cube.rotation.x += 0.01;        // X軸に回転  
  cube.rotation.y += 0.01;        // Y軸に回転  
  renderer.render(scene, camera); // 再描画  
}  
animate();                        // ループ開始
```



60 FPS

ブラウザの描画タイミングに同期（通常 秒60回）



省電力

タブが非表示の間は自動的に停止する

ウィンドウリサイズへの対応

画面サイズが変わっても歪まないよう、カメラとレンダラーを更新

```
window.addEventListener('resize', () => {  
  camera.aspect = innerWidth / innerHeight; // 比率を更新  
  camera.updateProjectionMatrix();           // 投影行列を再計算  
  renderer.setSize(innerWidth, innerHeight); // 描画サイズ更新  
});
```



忘れると何が起こる？

updateProjectionMatrix() を呼ばないと、画面が伸びて立方体が楕円に歪む

よくあるつまずきポイント

Q

画面が真っ黒で何も映らない

camera.position.z を設定したか確認。原点だとカメラが立方体の中にいる

Q

立方体が動かない

animate() を呼び忘れ。または render がループ外にある

Q

リサイズで歪む

updateProjectionMatrix() と setSize() の両方が必要

Q

Basic材質が真っ黒

ライトが無い場合は Normal か Basic 材質で確認

▶ HANDS-ON

回転する立方体を表示しよう

Step 1

Scene ・ Camera ・ Renderer を用意

前回のプロジェクトの main.js に記述

Step 2

BoxGeometry でメッシュ作成

MeshNormalMaterial と組み合わせて scene.add

Step 3

animate() ループを実装

rotation.x / y を毎フレーム加算

Step 4

リサイズ対応を追加

resize イベントでカメラ・レンダラー更新

Step 5

ブラウザで確認

画面中央で立方体がなめらかに回転

ゴール: 画面中央でカラフルな立方体が連続回転し、リサイズしても崩れない

STEP 1

Scene ・ Camera ・ Renderer を用意

main.js を開き、Three.js の3要素をセットアップする

```
import * as THREE from 'three';

const scene = new THREE.Scene();
scene.background = new THREE.Color(0x0f172a);

const camera = new THREE.PerspectiveCamera(
  75, innerWidth / innerHeight, 0.1, 1000);
camera.position.z = 5;

const renderer = new THREE.WebGLRenderer({ antialias: true });
renderer.setSize(innerWidth, innerHeight);
document.body.appendChild(renderer.domElement);
```



書く場所

前回の main.js の中身を全て消してスタート



期待結果

ダークブルーのキャンバスが全画面表示される

STEP 2

BoxGeometry でメッシュ作成

MeshNormalMaterial と組み合わせて scene.add する

```
// --- Step 1 のコードの下に追加 ---  
  
const geometry = new THREE.BoxGeometry(1, 1, 1);  
const material = new THREE.MeshNormalMaterial();  
const cube = new THREE.Mesh(geometry, material);  
scene.add(cube);  
  
renderer.render(scene, camera);
```



ポイント

renderer.render() を1回呼ぶことで静止画として描画



期待結果

画面中央に静止した立方体が1つ表示される

STEP 3

animate() ループを実装

requestAnimationFrame で毎フレーム回転させる

```
// --- renderer.render() を削除し、以下に置換 ---
```

```
function animate() {  
  requestAnimationFrame(animate);  
  cube.rotation.x += 0.01;  
  cube.rotation.y += 0.01;  
  renderer.render(scene, camera);  
}  
animate();
```



ポイント

rotation の値を変えると回転速度が変わる



期待結果

立方体がX軸・Y軸に連続回転し始める

STEP 4

リサイズ対応を追加

ウィンドウサイズ変更時にカメラとレンダラーを更新する

```
// --- animate() の後に追加 ---  
  
window.addEventListener('resize', () => {  
  camera.aspect = innerWidth / innerHeight;  
  camera.updateProjectionMatrix();  
  renderer.setSize(innerWidth, innerHeight);  
});
```

✔ ポイント

updateProjectionMatrix() を忘れると映像が歪む

✔ 期待結果

ブラウザ幅を変えても立方体が正しく表示される

STEP 5

ブラウザで確認・微調整

npm run dev で開発サーバーを起動し動作をチェック

```
$ npm run dev
```

```
# ブラウザで http://localhost:5173 を開く
```

```
# チェックリスト:
```

- # 1. 立方体が画面中央に表示されるか
- # 2. カラフルに回転しているか
- # 3. ウィンドウリサイズで歪まないか
- # 4. コンソールにエラーがないか



確認項目

立方体が中央に表示・回転・リサイズ対応を確認



困ったら

スライド16のデバッグガイドを参照

まとめ & 次回予告

今日のまとめ

- ✓ Scene / Camera / Renderer の三要素
- ✓ Mesh = Geometry + Material
- ✓ requestAnimationFrame でアニメーション
- ✓ リサイズ時はカメラとレンダラーを更新

次回: SESSION 03

マテリアルとライトの使い方

各種マテリアルとライトの違いを理解し、用途に応じて使い分けられるようになります